

SYSTEM ARCHITECTURE SPECIFICATION

NetOps

Distributed Multi-Node Network Architecture

Split-horizon DNS resolution, TLS 1.3 transport security, Layer 7 reverse proxying and load balancing, RFC 7232 conditional caching, and deep packet inspection, spread across three physical macOS machines on a shared local area network.

dnsmasq | Nginx | TLS 1.3 | RFC 7232 | Wireshark

PHYSICAL NODES

MAC 1
Ishan
10.7.11.142

Edge Gateway · Nginx
TLS termination · :443

MAC 2
Nikith
10.7.6.222

DNS · dnsmasq
Backend Worker B

MAC 3
Harshit
10.7.11.234

Client Testbench
Backend Worker A

Contents

1. Architectural Overview 3

2. Hardware, Host & Interface Matrix 4

3. End-to-End Request & Protocol Lifecycle 5

4. Subsystem Specifications 6

4.1 Domain Name Resolution Subsystem 6

4.2 Ingress & Reverse Proxy Subsystem 6

4.3 Transport Layer Security & PKI Subsystem 7

4.4 Application Worker Subsystem 8

5. OSI Model Layer Mapping 9

6. Packet Analysis & Wireshark Filter Matrix 9

7. Fault-Tolerance, Failover & Edge Cases 10

AT A GLANCE

3	4	TLS 1.3	60s
Physical macOS nodes	Architecture tiers	Encrypted ingress	Cache max-age

What this document covers. An enterprise-grade, distributed multi-node network architecture demonstrating split-horizon DNS resolution, transport layer security (TLS 1.3), Layer 7 reverse proxying and load balancing, RFC 7232 HTTP conditional caching, and deep packet inspection across physical machines.

1 Architectural Overview

The **NetOps** system distributes network responsibilities across three independent physical macOS machines operating over a shared local area network (LAN). The design isolates concerns into distinct tiers:

TIER 1

Name Resolution Tier

Private split-horizon DNS using `dnsmasq`.

TIER 2

Perimeter Ingress & Proxy Tier

Edge gateway: TLS termination and reverse load balancing using `Nginx`.

TIER 3

Application & Storage Tier

Microservices: redundant Python HTTP application workers with cryptographic cache validation.

TIER 4

Client & Verification Tier

Testbench issuing DNS queries, TLS handshakes, and packet analysis via `Wireshark`.

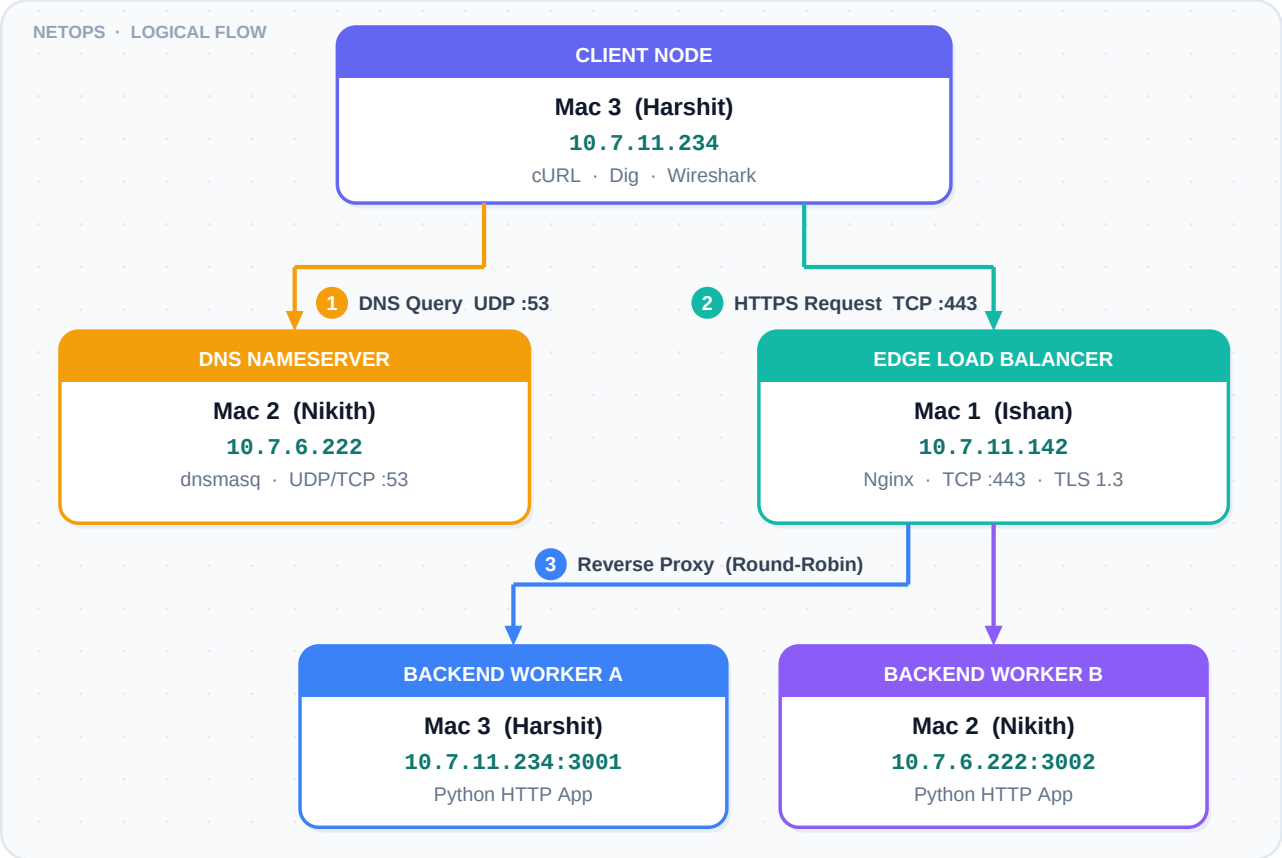


Figure 1. Logical request flow: DNS lookup, HTTPS ingress, and round-robin reverse proxying to backend workers.

2 Hardware, Host & Interface Matrix

MAC 1

Ishan

`10.7.11.142`

Nginx Reverse Proxy & Load Balancer
TCP `443` · TLS 1.3

MAC 2

Nikith

`10.7.6.222`

dnsmasq Nameserver + Backend Worker B
UDP/TCP `53` · TCP `3002`

MAC 3

Harshit

`10.7.11.234`

Backend Worker A + Client Testbench
TCP `3001` · cURL, Dig, Wireshark

Interface matrix

Host	Member	Assigned IP	Open Ports	Protocol	Running Service	Primary Configuration File
Mac 1	Ishan	`10.7.11.142`	TCP `443`	TLS 1.3 / HTTPS	Nginx Reverse Proxy & Load Balancer	`Config/nginx.conf`
Mac 2	Nikith	`10.7.6.222`	UDP/TCP `53` TCP `3002`	DNS HTTP	dnsmasq Local Nameserver Backend Worker B	`Config/dnsmasq.conf` `Backends/backend_B.py`
Mac 3	Harshit	`10.7.11.234`	TCP `3001` Ephemeral	HTTP Various	Backend Worker A Client (cURL, Dig, Wireshark)	`Backends/backend_A.py` N/A (Client Terminal)

Network placement

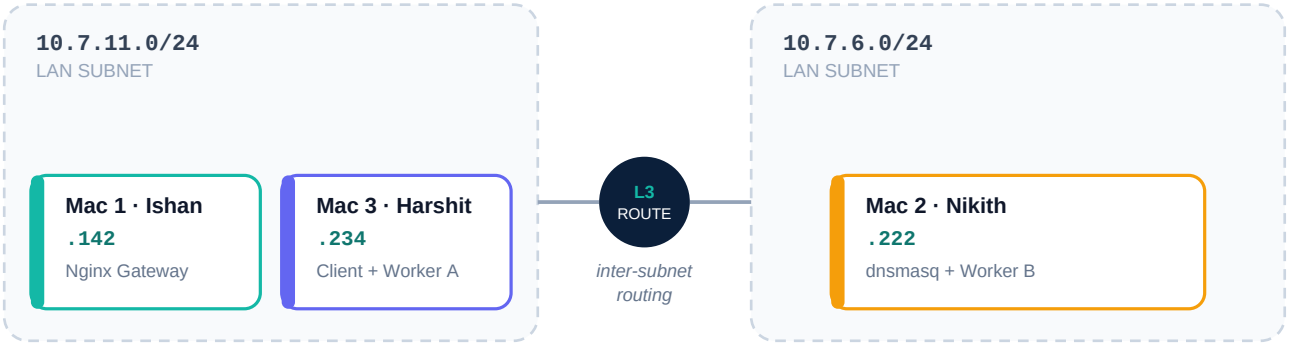


Figure 2. Host placement across the two LAN subnets; Mac 1 and Mac 3 share a subnet, Mac 2 sits on a routed neighbor.

3 End-to-End Request & Protocol Lifecycle

A single client request traverses four phases: split-horizon DNS resolution, a TCP three-way handshake with a TLS 1.3 negotiation, an encrypted Layer 7 request that Nginx balances round-robin across both workers, and finally a conditional cache validation that returns a zero-length **304 Not Modified**. Solid arrows are requests; dashed arrows are responses.

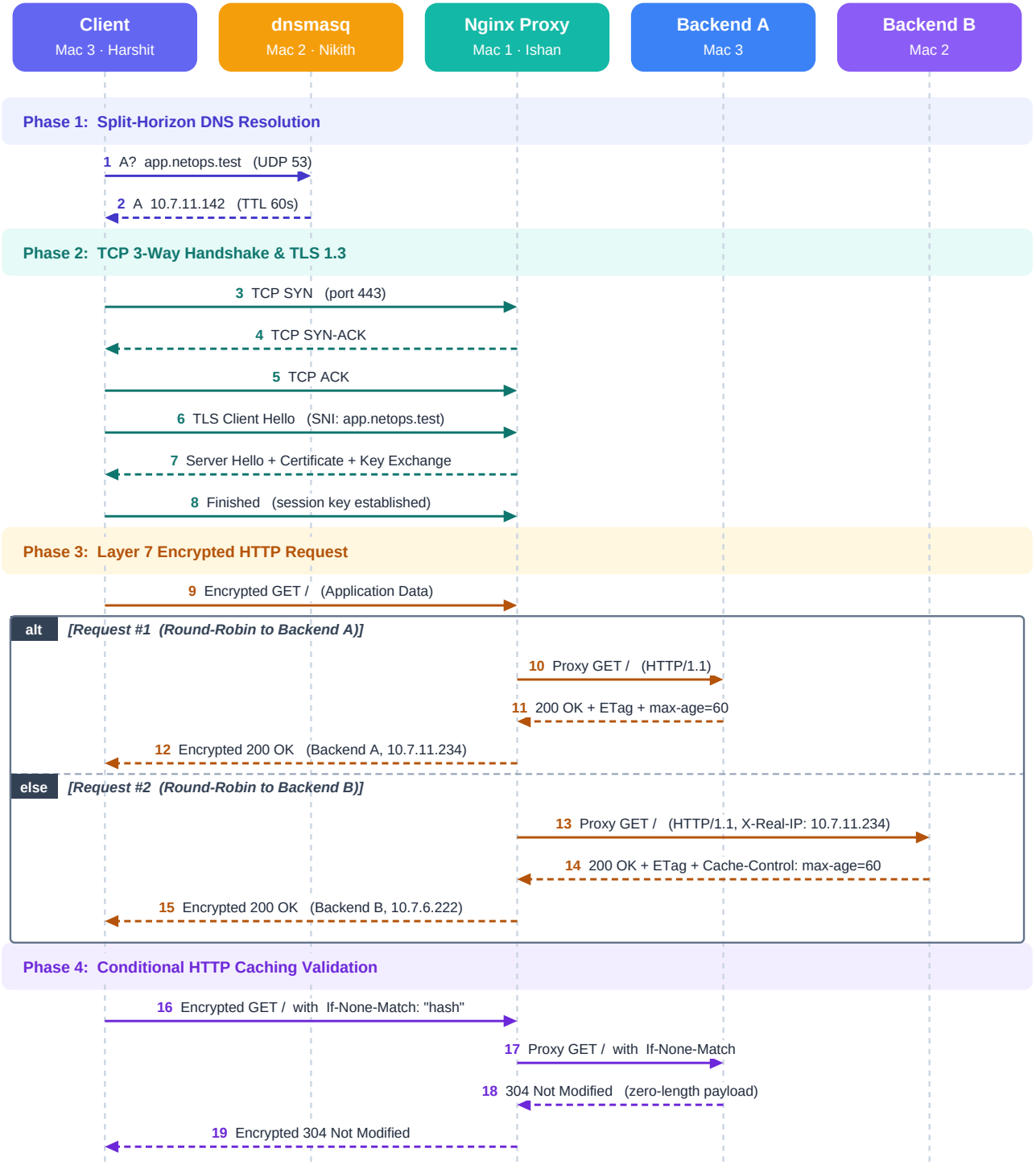


Figure 3. Sequence diagram of the complete NetOps request lifecycle.

4 Subsystem Specifications

4.1 Domain Name Resolution Subsystem

`Config/dnsmasq.conf`

Engine	`dnsmasq` lightweight DNS caching daemon.
Bind IP & Port	`10.7.6.222:53` on Mac 2.

Authoritative mapping

`dnsmasq.conf`

```
address=/app.netops.test/10.7.11.142
```

Forwarding & isolation

Directive	Effect
`no-resolv`	Bypasses default macOS upstream DNS configurations from `/etc/resolv.conf`.
`no-hosts`	Suppresses local hostfile pollution.
`server=1.1.1.1` & `server=8.8.8.8`	Upstream fallback recursion for non-project queries.
`cache-size=1000`	In-memory LRU DNS response caching.
`log-queries`	Emits query-by-query traces to stdout/syslog for Wireshark cross-verification.

4.2 Ingress & Reverse Proxy Subsystem

`Config/nginx.conf`

Engine	`Nginx` (HTTP / reverse proxy engine).
Listening interface	`0.0.0.0:443 ssl` on Mac 1 (`10.7.11.142`).
Load balancing	Weighted Round-Robin across the upstream pool.

`nginx.conf`

```
upstream backends {
    server 10.7.11.234:3001;  # Backend A (Mac 3)
    server 10.7.6.222:3002;   # Backend B (Mac 2)
}
```

Client IP & header preservation

Directive	Purpose
`proxy_set_header Host $host`	Retains client requested hostname.
`proxy_set_header X-Real-IP $remote_addr`	Conveys true client IP address.
`proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for`	Builds provenance chain.
`proxy_set_header X-Forwarded-Proto $scheme`	Conveys encryption context (`https`).

4.3 Transport Layer Security & PKI Subsystem

`mkcert · private certificate authority`

Tooling	`mkcert` (private Certificate Authority management).
CA root	Trusted via macOS System Keychain on all participant machines (`security add-trusted-cert`).
Cipher suite	Modern TLS 1.2 / TLS 1.3 ephemeral ECDHE key exchanges with AES-GCM encryption.

Leaf certificate

leaf certificate paths

```
# certificate
/Users/ishanmaheshwari1/Desktop/CNProject/app.netops.test.pem
# private key
/Users/ishanmaheshwari1/Desktop/CNProject/app.netops.test-key.pem
```

Subject Alternative Names (SAN)

`app.netops.test` `10.7.11.142` `localhost`

4.4 Application Worker Subsystem

`Backends/backend_A.py` · `Backends/backend_B.py`

Runtime	Python 3 standard library `http.server.HTTPServer`.

Payload schema

`application/json`

```
{
  "ip":       "10.7.11.234",
  "mac_name": "harshit",
  "backend":  "A",
  "status":   "ok"
}
```

RFC 7232 HTTP conditional caching architecture

1

Compute a deterministic digest

Computes an MD5 digest over the payload bytes.

```
ETag = MD5( Payload )
```

2

Evaluate the client conditional header

A matching validator short-circuits the response.

```
If-None-Match == ETag ==> HTTP 304 Not Modified
```

3

Emit cache header directives

```
ETag: "<hash>"
Cache-Control: max-age=60
X-Backend: A   # or B
```

5 OSI Model Layer Mapping

How each layer of the OSI model is exercised by the components in the NetOps architecture.

Layer	Protocol / Standard	Component Involved	Function in NetOps Architecture
7 Application	**HTTP/1.1, HTTP/2, DNS**	cURL, Nginx, Python Workers, dnsmasq	REST API JSON transport, DNS resource records, HTTP conditional caching headers.
6 Presentation	**TLS 1.3, JSON, MD5/SHA**	mkcert, OpenSSL, Python hashlib	Data serialization, asymmetric/symmetric encryption, integrity validation.
5 Session	**TLS Session / TCP Keep-Alive**	OpenSSL, Nginx Worker Processes	Session establishment, TLS ticket resumption, session termination.
4 Transport	**TCP, UDP**	macOS Kernel TCP/IP Stack	Port 53 UDP query/response, Port 443 TCP 3-way handshake (SYN, SYN-ACK, ACK), flow control.
3 Network	**IPv4, ICMP, ARP**	macOS Routing Table, Wireshark	Inter-subnet routing (10.7.11.0/24 to 10.7.6.0/24), ping echo diagnostics.
2 Data Link	**IEEE 802.3 / 802.11 (Wi-Fi)**	Network Interface Cards (NIC)	MAC address resolution, framing, collision prevention.
1 Physical	**RF / Cat6 Cabling**	Wi-Fi Access Point / Switch	Raw bitstream transmission across physical medium.

6 Packet Analysis & Wireshark Filter Matrix

To capture and inspect network operations during live execution, apply the following display filters in Wireshark on **Mac 3 (Client)**.

Inspection Objective	Protocol	Wireshark Display Filter	Verification Criteria
DNS Resolution	DNS (UDP)	`dns.qry.name == "app.netops.test"`	Query sent to `10.7.6.222:53`; response answers with `10.7.11.142`.
TCP Three-Way Handshake	TCP	`tcp.port == 443 && (tcp.flags.syn == 1 || tcp.flags.ack == 1)`	Sequence starts with [SYN], answered by [SYN, ACK], finalized with [ACK].
TLS 1.3 Cryptographic Handshake	TLS	`tls.handshake.type == 1 || tls.handshake.type == 2`	Client Hello showing Server Name Indication: app.netops.test, followed by Server Hello.
Encrypted HTTP Application Data	TLS Record	`tls.record.content_type == 23`	Application Data packets are fully encrypted binary payloads (plain HTTP headers invisible).
Backend Communication	HTTP (Plain)	`tcp.port == 3001 || tcp.port == 3002`	Unencrypted internal proxy traffic between Nginx (10.7.11.142) and Backends.
HTTP Conditional Caching	HTTP	`http.response.code == 304 || http.response.code == 200`	Demonstrates bandwidth savings when client sends `If-None-Match`.

7 Fault-Tolerance, Failover & Edge Cases

Expected behavior when components fail or the client reaches for something that does not exist.

1 Backend Server Failure

If `backend_A.py` is terminated, Nginx detects the TCP connection refusal (`ECONNREFUSED`) or timeout and routes subsequent incoming requests to `backend_B.py` without dropping client connections.

2 DNS Unavailability / NXDOMAIN

Querying standard nameservers without targeting `10.7.6.222` yields `NXDOMAIN`, demonstrating true DNS split-horizon isolation.

3 Invalid Ports / Unassigned Endpoints

Requesting closed ports triggers immediate TCP `[RST, ACK]` packets, validating active transport layer rejection.

4 Invalid Subnet Addresses

Requesting unrouted IP addresses triggers ICMP `Destination Host Unreachable` from the default gateway.

Takeaway. Each failure mode surfaces at a different layer, from Layer 3 (ICMP unreachable) and Layer 4 (TCP reset) up to Layer 7 (Nginx failover between backend workers), so every tier of the architecture can be verified independently with Wireshark.